

Chapter 4 Assignment

1. Is JSX mandatory for React?

Ans: - No, JSX is not mandatory in React. React can work without JSX using `React.createElement()`. JSX is just a syntactic sugar that makes code easier to write and read.

- Behind the scene the JSX transpiles the code into `React.createElement` using Babel.
- Eg. `const element = <h1>Hello</h1>;` get converted in to `const element = React.createElement("h1", null, "Hello");`

2. Is ES6 mandatory for React?

Ans:- No, ES6 is not mandatory for React, but modern React development heavily uses ES6 features because they make the code cleaner, simpler, and easier to maintain.

- We use ES6 because, It makes code shorter, cleaner, easier to maintain, and easier to understand.
- Many React Concepts rely on ES6 syntax like Components, Hooks, Props destructuring, Module imports/ exports.

3. `{TitleComponent}` vs `<TitleComponent />` vs

`<TitleComponent></TitleComponent>` in JSX

Ans: To understand this concept let's use a functional Component.

```
const TitleComponent = () => {
```

```
  return <h1> Hello </h1>;
```

```
};
```

- When we write like this `{TitleComponent}`
- We are passing a function itself, not rendering the Component.
- The Output will be `() => {return <h1> Hello </h1>}`
- Which is not useful, nothing meaningful on the Screen.

```
<TitleComponent />
```

- When we write like this, we are rendering the component.
- It will give the output Hello on the Screen.
- This is also known as self closing tag.

```
<TitleComponent></TitleComponent>
```

- When we write like this, its also a component Rendering.
- The difference is that we wrote it in the format of Opening and closing Tag.
- This is mostly used when the Component have Children to render.

3. How can I write comments in JSX?

Ans: In JSX, comments are written using JavaScript comment syntax

Inside curly braces, like `{/* comment */}`. HTML comments do not Work in JSX.

Eg. `{/* This is a JSX comment */}` → Single Line Comment.

```
{
  /* This is a
    Multi- line JSX comment
  */
}
```

4. What is `<React.Fragment>``</React.Fragment>` and `<></>` ?

Ans.- `React.Fragment` and `<></>` are used to group multiple JSX elements without adding extra HTML elements to the DOM.

- The short syntax is looks cleaner, But when we need To use props in the fragment like key, we have to use like This,

```
Items.map((item) => (
  <React.Fragment key={item.id}>
    <h1>{item.name} </h1>
  </React.Fragment>
));
```

- Otherwise we can write like this.

```

Const App = () => {
  Return (
    <>
      <h1> Hello </h1>
      <p> World </p>
    </>
  );
};

```

5. What is Virtual DOM? & 7. What is Reconciliation in React?

Ans: -> DOM is a Document Object Model, which is a Tree like structure of an React Elements.

- > when we console.log the component we can see that it is a JS Object.
- > So, Virtual DOM is the copy of that Object.
- > when we use useState() we are monitoring the changes in the state.
- > exactly when we call setState statement, the diffing algorithm gets trigger..
- > Diffing algorithm is one of the react feature which keeps on checking the virtual DOM previous copy and the copy created when setState call.
- > Find out the changes and make that only changes in that actual Node of a DOM.
- > This process is called Reconciliation.
- > So, the simple words we can say that, without make changes in a entire actual DOM every time, React can update the desire Node..
- > This makes the Process faster.

8. What is React Fiber?

Ans: - React Fiber is the reconciliation engine introduced in React 16.

- It improves rendering performance by breaking rendering work
Into smaller units, prioritizing updates, and allowing
Rendering to pause and resume.

- It Makes the React application smooth and more responsive.

9. Why we need keys in React? When do we need keys in React?

Ans: - In React keys helps to find out exact which item is changed, Added, removed or restore.

- The is required for the diffing process due to which Reconciliation happens.
- Without key, React may re-render unnecessary elements, Update wrong elements, cause performance issues, or create UI bugs.

10. Can we use index as keys in React?

Ans: - No, we can not use index as a key, because it may Cause the error when items are inserted, deleted or reordered.

- Always use unique IDs from database.

11. What is props in React? Ways to use props in React?

Ans:- Props are similar to the passing the argument in a JavaScript function.

- Props in React are used to pass data from parent components to the child components.
- They are read-only and help make components reusable and dynamic.
- Props can be passed as values, objects, functions or even children components.

Ways to use Props in React..

1. Using props object

```
Eg. Function User(props) {  
    Return <h1> {props.name} </h1>;  
}
```

2. Destructuring Props in Parameters

```
Eg. Function User ( { name, age } ) {  
    Return (  
        <h1> { name } – { age } </h1>  
    );  
}
```

It can be use like;

```
<User name = "Anuja" age={22} />
```

3. Destructuring Inside Component

```
Function User(props) {  
  Const { name, age } = props;  
  Return <h1> { name } </h1>;  
}
```

4. Passing Multiple Props

```
<User name = "Anuja" city = "Navi Mumbai" age = {22} />
```

Receiving:

```
Function User(props) {  
  Return (  
    <div>  
      <h1>{ props.name } </h1>  
      <p> { props.city } </P>  
      <p> { props.age } </P>  
    </div>  
  );  
}
```

5. Passing Objects as Props

```
Const user = { name: "Anuja",  
  Age : 22  
};
```

```
<User data = {user} />
```

Access:

```
Function User(props) {  
  Return <h1> {props.data.name} </h1>;  
}
```

6. Passing Functions as Props.

Parents sends function to child.

```
Function App() {  
  Const greet = () => {  
    Alert("Hello");  
  };  
  Return <Button handleClick = {greet} />  
}
```

Child:

```
Function Button(props) {  
  Return <button onClick = {props.handleClick} >  
    Click  
  </button>;  
}
```

7. Using children Prop

Anything written between component tags becomes children.

Eg. <Card>
 <h1>Hello</h1>
 </Card>

Child component:

```
function Card(props) {  
  return <div>{props.children}</div>;  
}
```

12. What is a Config Driven UI ?

Ans: A Config Driven UI is a design approach where the UI is dynamically generated using configuration data instead of hardcoded layouts. The frontend reads data from APIs or JSON configs and renders components accordingly, making applications more flexible, scalable, and easier to update.

Eg. On Food Order Apps like Swiggy, We can see the different Offers available for the different cities..

Here the UI uses dynamically configured Data instead of Hard coded data.